K-MOSA 지향 독립형 Modular GCS 상세 소프트웨어 아키텍처 설계서

MOSA / UCS / OMS / UCI 기반 개념 정리부터
클래스 관계 · 책임 · 입출력 · 스레드 구조 · 첫 구현 진입점까지

Architecture Baseline v0.1

작성 기준일: 2026-09-05
프로젝트 코드명: KM-GCS

문서 목적과 범위

이 문서는 지금까지 논의한 미국 MOSA, UCS, OMS/UCI, FACE/SOSA/HOST 의 관계와 K-MOSA 적용 방향을 하나의 기준선으로 정리하고, 이를 바탕으로 QGroundControl 을 포크하지 않는 독립형 Ground Control System(KM-GCS)의 상세 소프트웨어 아키텍처를 정의한다. 특히 이후 구현 단계에서 해석이 갈리지 않도록 계층, 모듈, 클래스 책임, 클래스 간 의존성, 데이터 모델, 입출력, 이벤트 흐름, 스레드 경계를 명시한다.

본 문서는 미국 표준의 직접 구현 또는 적합 인증 문서가 아니다. 공개적으로 확인 가능한 미국 DoD/SAE 자료에서 아키텍처 원칙을 추출하고, PX4·ArduPilot·MAVLink·ROS 2·DDS·DroneCAN·영상 스트림을 사용하는 소형/전술 UAS 생태계에 맞춘 경량 K-MOSA 지향 Control Segment Architecture 를 제안하는 설계 기준서다.

- 범위: MOSA/K-MOSA 배경, 미국 Open Architecture 표준의 역할 분리, KM-GCS 시스템/소프트웨어 아키텍처, 상세 클래스 설계, 스레드/동시성 모델, 오류/보안/시험 방향, MVP 와 단계별 구현 로드맵.
- 초기 구현 우선순위: PX4/ArduPilot SITL 또는 실제 FC 와 UDP MAVLink 연결 후 HEARTBEAT, ATTITUDE, POSITION, GPS, BATTERY, ARM/DISARM, Flight Mode, 지도 표시.
- 장기 확장: Mission semantic model, Payload plugin, Capability discovery, Architecture View, Interface Profile, Conformance Test, DDS/ROS 2/UCI adapter, Multi-Vehicle/MUM-T.

목차

1. MOSA 와 미국 Open Architecture 체계
2. K-MOSA 와 본 프로젝트의 위치
3. KM-GCS 설계 목표와 원칙
4. 전체 시스템 아키텍처
5. 소프트웨어 계층 아키텍처
6. Canonical Domain Model
7. 상세 클래스 관계
8. 클래스별 책임·입력·출력
9. Event Bus 와 데이터 흐름
10. 스레드 및 동시성 구조
11. MAVLink/PX4/ArduPilot 분리
12. Mission/Parameter/Payload 구조
13. Plugin/Capability/Architecture View
14. ROS 2/DDS 의 위치
15. Interface Profile 과 Conformance
16. 오류 처리·관측성·보안 경계
17. 저장소/빌드 구조
18. MVP 구현 순서
19. 시험 전략
20. 첫 코드 진입용 인터페이스 골격
21. 향후 로드맵

부록 A. 미국 표준 대응표

1. MOSA 와 미국 Open Architecture 체계

1.1 MOSA 의 본질

미국 DoD 의 Modular Open Systems Approach(MOSA)는 하나의 프로토콜이나 단일 기술 표준이 아니라, 시스템 수명주기 전체에서 경쟁적·경제적인 획득과 유지, 기술 삽입, 부품 교체, 벤더 종속 완화를 달성하기 위한 통합 비즈니스·기술 전략이다. 공식 설명은 modular, loosely coupled, highly cohesive structure 와 표준 인터페이스 및 검증 가능한 conformance 를 강조한다.

그림 1-1. MOSA 를 본 프로젝트 관점에서 요약한 구성

```
MOSA
  = Modular Design
  + Open / Standardized Key Interfaces
  + Replaceable Components
  + Verifiable Conformance
  + Version / Governance
  + Business & Data-Rights Strategy
```

따라서 MAVLink, DDS 또는 ROS 2 를 쓴다는 사실만으로 MOSA 가 되는 것은 아니다. 이들은 MOSA 아키텍처를 구현하기 위한 후보 기술일 뿐이며, 시스템의 모듈 경계와 계약, 교체성, 적합성 시험 및 진화 관리가 함께 설계되어야 한다.

1.2 미국 Open Architecture 표준은 일렬 계층이 아니다

표준/접근	주요 초점	UAS 에서의 대표 위치	KM-GCS 에서의 참고점
MOSA	상위 획득/설계 전략	전체 UAS 체계	전체 설계 원칙
FACE	항공 SW portability / modularity	Avionics software	Plugin/API 분리, 구현 종속 완화
SOSA	C4ISR 센서/처리 HW·SW	센서/임무처리	Payload/Sensor 확장 시 참고
HOST	Embedded HW openness	Mission computer HW	향후 임무컴퓨터 HW 모듈화
OMS	Mission services 와 subsystem interface	Mission computer / payload	Mission/Payload service 구조
UCI	Mission-level machine-to-machine C2	C2/mission exchange	향후 UCI adapter
UCS	Unmanned Systems Control Segment architecture	GCS / Control Segment	KM-GCS 의 직접적 아키텍처 참고

1.3 UCS Architecture Library 에서 가져올 핵심

SAE AS6512B 는 UCS Architecture Revision B 의 Architecture Description 이며 AS6513B 와 AS6518B 를 핵심 연계 문서로 명시한다. AS6518B Architecture Model 은 다른 UCS model/product 의 authoritative source 로 설명되고, AS6513B 는 UCS product 의 conformance requirement 를 정하는 authoritative specification 이다. AS6522B 는 architecture model 및 사용자 extension 의 technical governance 를 다룬다.

그림 1-2. UCS 에서 본 프로젝트가 참고하는 구조

```
SAE UCS Architecture

AS6512B  Architecture Description
```

```
|
+-- AS6518B  Architecture Model (authoritative model)
+-- AS6513B  Conformance Specification
+-- AS6522B  Technical Governance / extensions
+-- AIR6515 etc.  ICD/model-related supporting material
```

본 프로젝트는 SAE UCS 를 그대로 복제하거나 "UCS conformant"라고 주장하지 않는다. 대신 Architecture Model 중심의 설계, 서비스/정보 모델 분리, 명시적 인터페이스, conformance test, extension governance 라는 구조적 원리를 경량화하여 적용한다.

2. K-MOSA 와 본 프로젝트의 위치

2026 년 설립된 K-MOSA 연구센터는 국방 무인체계의 한국형 MOSA 구현을 목표로 모듈 인터페이스 설계, MUM-T 아키텍처, AI 자율기능 모듈, 시험·인증 방법론, 정책·표준화 등을 연구 영역으로 제시하고 있다. 즉 국내에서도 단순한 하드웨어 모듈화가 아니라 인터페이스와 시험·인증까지 포함한 체계적 접근이 중요해지고 있다.

2.1 우리가 풀려는 구체적 문제

- 기존 GCS 가 특정 autopilot, 특정 protocol, 특정 payload semantics 와 강하게 결합될 경우 비행체 또는 임무장비 교체 때 GCS 수정 범위가 커진다.
- MAVLink 메시지를 그대로 UI/비즈니스 로직에 노출하면 MAVLink 가 사실상 GCS 내부 표준이 되어 다른 프로토콜(DDS, UCI, ROS 2)의 삽입이 어려워진다.
- 모듈이 교체 가능하려면 단순 통신 연결뿐 아니라 capability, semantic data model, version, timing, behavior, conformance 가 명시되어야 한다.
- 따라서 GCS 자체가 MOSA 의 Control Segment 축으로 설계되어야 하며, 실제 연결된 UAS architecture 를 이해하고 검증할 수 있어야 한다.

2.2 프로젝트의 최종 정의

KM-GCS 는 "Protocol-independent, Vehicle-independent, Payload-modular, Architecture-aware, Capability-aware Ground Control System"을 목표로 한다. 미국 UCS 의 architecture/conformance/governance 사고방식을 참고하면서도 PX4, ArduPilot, MAVLink, ROS 2, DDS, DroneCAN 등 현재의 오픈 UAS 생태계를 활용한다.

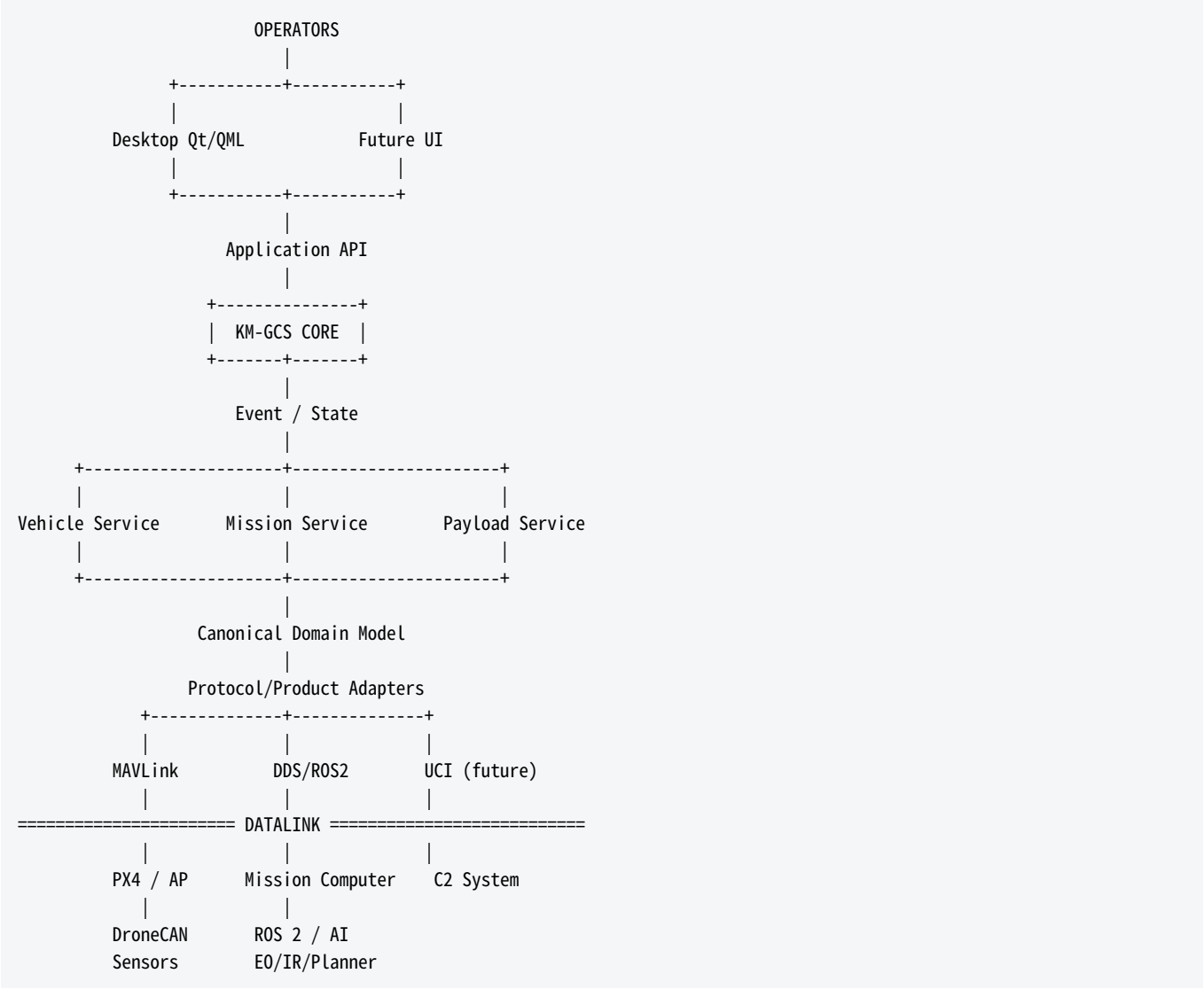
3. KM-GCS 설계 목표와 핵심 원칙

ID	설계 원칙	구현 의미
P1	Core 는 protocol-independent	Core API 에 mavlink_message_t, DDS type, ROS msg 를 노출하지 않는다.
P2	UI 와 Core 분리	QML UI 는 Application/Service API 만 본다. Core 는 QML 을 모른다.
P3	Link 와 Protocol 분리	UDP/Serial/TCP 와 MAVLink/DDS/UCI 를 독립 abstraction 으로 둔다.
P4	Protocol 과 Product semantics 분리	MAVLinkAdapter 와 PX4Adapter/ArduPilotAdapter 를 분리한다.
P5	Canonical Domain Model	모든 외부 데이터는 내부 표준 semantic model 로 변환한다.
P6	Event-driven	수신 path 는 event/message 기반으로 서비스와 UI 의 시간 결합을 낮춘다.
P7	Plugin/Capability 기반	Autopilot, payload, mission 기능을 plugin 과 contract 로 확장한다.

ID	설계 원칙	구현 의미
P8	Conformance built-in	인터페이스 존재 여부뿐 아니라 semantic/timing/behavior 를 시험한다.
P9	Replaceability 검증	PX4→ArduPilot 또는 payload A→B 교체 시 Core 변경 최소화를 실험한다.
P10	Gradual complexity	v0.1 은 MAVLink 기본 GCS 로 시작하고 추상화가 실제로 작동함을 먼저 검증한다.

4. 전체 시스템 아키텍처

그림 4-1. KM-GCS 와 Air Segment 를 포함한 전체 시스템 구조



4.1 Air Segment

- Autopilot: PX4 또는 ArduPilot. 실시간 자세/위치 제어, estimator, actuator output, 기본 flight safety 담당.
- Mission Computer: ROS 2 기반 perception, AI, path planning, tracking, higher-level autonomy 담당.
- Payload: EO/IR, gimbal, radar, custom mission payload. 제어 채널과 영상 채널을 분리할 수 있다.

- Datalink: Wi-Fi/HaLow/Ethernet/RF 등을 물리/링크 기술로 보고, 상위 protocol semantics 와 분리한다.

4.2 Control Segment

KM-GCS 는 차량 운용 UI 만 제공하는 애플리케이션이 아니라 Vehicle, Mission, Payload, Parameter, Health, Capability 및 Interface Architecture 를 통합하는 Control Segment 다. Architecture View 와 Interface/Conformance 도구가 일반 GCS 와의 핵심 차별점이다.

5. 소프트웨어 계층 아키텍처

그림 5-1. 소프트웨어 계층

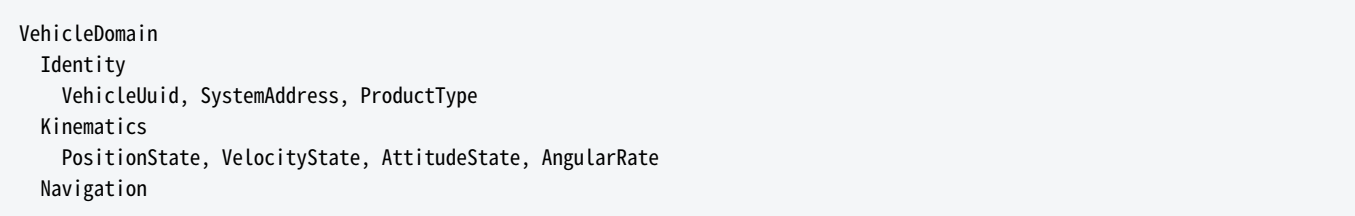


의존성 방향은 항상 위 계층이 아래 계층의 "추상 인터페이스"에 의존하도록 한다. 저수준 protocol type 이 service/UI 까지 올라오지 않도록 Translation/Adapter boundary 를 강제한다.

6. Canonical Domain Model

Canonical Domain Model 은 KM-GCS 의 심장이다. 외부 세계의 protocol-specific representation 을 KM-GCS 내부에서 안정적으로 사용하는 semantic representation 으로 변환한다. 같은 위치·자세·배터리라는 개념을 MAVLink, DDS, ROS 2 가 서로 다른 타입으로 표현해도 Core 는 하나의 모델만 본다.

그림 6-1. Canonical Vehicle Domain Model



```
  GnssState, HomeState, NavigationState
Flight
  ArmedState, FlightMode, VtolState
Power
  BatteryState, PowerSystemState
Health
  HealthStatus, Fault, ComponentHealth
Mission
  Mission, MissionTask, MissionExecutionState
Payload
  PayloadState, CameraState, GimbalState, TargetTrack
Capability
  CapabilitySet, InterfaceProfileRef
```

6.1 좌표계와 단위

모든 domain value 는 가능한 한 좌표계와 단위를 명시한다. MAVLink NED/FRD, ROS ENU/FLU, 지리좌표 WGS84 가 섞이는 문제를 adapter boundary 에서 흡수한다. 내부적으로 자세는 quaternion 을 우선 canonical representation 으로 하고, UI 에서 Euler angle 로 파생한다.

타입	핵심 필드	정책
PositionState	lat/lon/alt 또는 local vector, frame, timestamp	WGS84/NED/ENU 를 명시하고 암묵 변환 금지
AttitudeState	Quaternion, angularVelocity, body frame, timestamp	Euler 는 파생값
VelocityState	Vector3, frame, timestamp	ground/air velocity semantic 을 명시
BatteryState	voltage/current/remaining/energy/status	raw protocol field 를 직접 노출하지 않음
FlightMode	canonical mode + vendor detail(optional)	PX4/AP mode mapping 은 product adapter 책임

7. 상세 클래스 관계

그림 7-1. 1 차 클래스 관계

```
GcsApplication
  |
  +-- GcsCore
       |
       +-- LinkManager ------------------+-- ILink
       |                                 +-- UdpLink
       |                                 +-- SerialLink
       |
       +-- ProtocolManager --------------+-- IProtocolAdapter
       |                                 +-- MavlinkProtocolAdapter
       |
       +-- VehicleRegistry --------------+-- VehicleContext [0..N]
       |                                      |
       |                                      +-- VehicleStateStore
       |                                      +-- VehicleCommandService
       |                                      +-- MissionService
       |                                      +-- ParameterService
```

```
      |                                    +-- HealthService
      |                                    +-- CapabilityService
      |                                    +-- PayloadRegistry
      |
      +-- AdapterRegistry ---------------+-- IAutopilotAdapter
      |                                  +-- Px4AutopilotAdapter
      |                                  +-- ArduPilotAdapter
      |
      +-- EventBus
      +-- PluginManager
      +-- TelemetryRecorder
      +-- ConformanceManager (later)

MavlinkProtocolAdapter
  -> MavlinkTranslator(s)
       -> DomainEvent
            -> EventBus
                 -> VehicleContext / services / recorder / analyzer
```

7.1 핵심 경계

- ILink 는 bytes 를 송수신할 뿐 MAVLink frame 을 알지 못한다.
- IProtocolAdapter 는 framing/parse/encode 를 담당하되 PX4 flight mode 의 의미를 결정하지 않는다.
- IAutopilotAdapter 는 PX4/ArduPilot 제품별 semantic 및 command mapping 을 담당하되 UDP/Serial 을 알지 못한다.
- VehicleContext 는 protocol object 를 보관하지 않고 canonical state 와 service 집합을 소유한다.
- QML ViewModel/Application Facade 는 domain/service API 만 보고 protocol class 에 직접 접근하지 않는다.

8. 클래스별 책임 · 입력 · 출력

클래스	책임	입력	출력	권장 스레드
GcsApplication	프로세스 bootstrap, Qt application lifecycle, configuration 로드	CLI/config, OS lifecycle	GcsCore 시작/종료, QML context 등록	UI/Main thread
GcsCore	핵심 서브시스템 조립 및 lifecycle orchestration	Application start/stop	Managers/registries 구동	Main/control thread
LinkManager	ILink 생성·등록·연결상태 관리	LinkConfig	Link events, raw byte streams	Main + link worker 소유
ILink	transport abstraction	send(bytes), connect/disconnect	bytesReceived, linkStateChanged	구현별 worker
UdpLink	QUdpSocket 기반 datagram 송수신	host/port, bytes	raw bytes + endpoint metadata	Network thread
SerialLink	QSerialPort 송수신	port/baud, bytes	raw bytes	Serial I/O thread
ProtocolManager	link 와 protocol instance 의 binding	Link stream, ProtocolConfig	parsed protocol events	Protocol worker
IProtocolAdapter	framing/parse/encode abstraction	raw bytes / domain-level send request	ProtocolEnvelope 또는 DomainEvent 후보	Protocol worker
MavlinkProtocolAdapter	MAVLink v1/v2 parser, sysid/compid routing, encode	raw bytes	MavlinkEnvelope / parse errors	Protocol worker
MavlinkTranslator	MAVLink message→canonical event 변환	MavlinkEnvelope	VehicleAttitudeUpdated 등 DomainEvent	Protocol worker, stateless 권장
VehicleRegistry	차량 발견·생성·삭제·lookup	Heartbeat/Discovery event	VehicleContext lifecycle	Core thread

클래스	책임	입력	출력	권장 스레드
			events	
VehicleContext	차량 단위 aggregate root	domain events, service commands	state change events, command results	Vehicle/core thread
VehicleStateStore	최신 canonical state 의 일관된 snapshot 보관	typed domain events	immutable/read snapshot, changed signals	Vehicle/core thread; read snapshots thread-safe
VehicleCommandService	arm/disarm/mode/takeoff/land 등 use-case	UI/API command	CommandHandle/Result	Core thread; async completion
IAutopilotAdapter	vendor-specific semantics abstraction	canonical command / vendor status	protocol command request / mapped canonical state	Core or adapter worker
Px4AutopilotAdapter	PX4 mode/capability/parameter semantics	canonical command, PX4-specific telemetry	MAVLink command specs, canonical mappings	Core thread
ArduPilotAdapter	ArduPilot semantics	same	same	Core thread
MissionService	mission CRUD, validate, compile, upload/download	MissionModel / execution command	MissionState, transfer progress/result	Mission worker for heavy compute + core state
ParameterService	parameter cache/metadata/read/write	parameter commands / protocol values	Parameter snapshot/change/result	Core + async transfer state machine
PayloadRegistry	payload discovery/instances	capability/discovery events	IPayloadService instances	Core thread
CapabilityService	capability merge/resolution	protocol capability + plugin manifest + probe	CapabilitySet change event	Core thread
PluginManager	plugin scan/load/unload/manifest validation	plugin paths/config	registered factories/capabilities	Main thread load; runtime calls controlled
EventBus	typed event fan-out, thread handoff 정책	DomainEvent	subscriber callbacks/queued dispatch	Cross-thread gateway
TelemetryRecorder	raw/domain telemetry 기록	raw frames/domain events	log file, metrics	Dedicated writer thread
ConformanceManager	profile 기반 syntax/semantic/timing/behavior 평가	InterfaceProfile + observed events	ConformanceReport	Dedicated test/analysis worker

9. Event Bus 와 데이터 흐름

9.1 수신 경로

그림 9-1. Telemetry 수신 path

```
Network Thread
  UdpLink::readyRead
      | raw datagram
      v
Protocol Worker
  MavlinkProtocolAdapter::consume(bytes)
      | MAVLink envelope
      v
  MavlinkTranslator
      | typed DomainEvent
      v
EventBus (queued handoff)
      |
      +--> VehicleRegistry / VehicleContext
      +--> TelemetryRecorder
      +--> Analyzer / Conformance observer
```

```
Core Thread
  VehicleStateStore update
      | stateChanged
      v
UI/Main Thread
  ViewModel / QML binding
```

9.2 송신 경로

그림 9-2. Command 송신 및 ACK path

```
UI/Main Thread
  ARM button
     |
Application API
     | ArmCommand(vehicleId)
     v
Core Thread
  VehicleCommandService
     | canonical command
     v
  Px4AutopilotAdapter
     | protocol-neutral command specification -> MAVLink mapping
     v
Protocol Worker
  MavlinkProtocolAdapter::encode(...)
     | bytes
     v
Network Thread
  UdpLink::send(bytes)

ACK 수신은 반대 path를 통해 CommandTracker를 완료한다.
```

9.3 Domain Event 예시

예시 9-1. Protocol-independent domain event

```
struct VehicleAttitudeUpdated {
    VehicleUuid vehicle;
    Quaternion orientation;
    Vector3 angularVelocity;
    ReferenceFrame frame;
    Timestamp sourceTime;
    Timestamp receiveTime;
};

struct VehiclePositionUpdated { ... };
struct BatteryUpdated { ... };
struct LinkQualityUpdated { ... };
struct CommandAcknowledged { ... };
```

10. 스레드 및 동시성 구조

첫 버전부터 모든 기능을 여러 스레드로 쪼개면 복잡성이 급증한다. 따라서 "I/O 와 장시간 작업만 worker 에 두고 domain state 는 단일 owner thread 에서 갱신한다"는 원칙을 적용한다. Qt 의 queued signal/slot 또는 명시적 EventBus queue 를 cross-thread boundary 로 사용한다.

스레드	주요 객체	금지/주의	이유
UI/Main	QGuiApplication, QML, ViewModel, Plugin load	blocking I/O 금지	UI 응답성
Core/Domain	VehicleRegistry, VehicleContext, StateStore, Services	외부 socket 직접 접근 금지	state mutation 직렬화
Network I/O	UdpLink/TcpLink	domain state 직접 수정 금지	socket event 처리
Serial I/O	SerialLink	domain 직접 수정 금지	serial blocking/latency 격리
Protocol Worker	MAVLink parse/translation	QML 접근 금지	burst telemetry 처리
Recorder	TelemetryRecorder	Core lock 장시간 점유 금지	disk I/O 격리
Mission/Analysis	경로 생성, large validation, conformance analysis	UI 직접 갱신 금지	CPU-heavy 작업 격리

10.1 State ownership 규칙

1. VehicleStateStore 의 mutable state 는 Core/Domain thread 만 수정한다.
2. 다른 스레드는 immutable snapshot 또는 event copy 를 사용한다.
3. Network/Protocol thread 는 VehicleContext 포인터를 직접 만지지 않고 event 를 게시한다.
4. UI 는 state snapshot/ViewModel property 만 읽고 command 는 service queue 로 보낸다.
5. Command completion 은 future/callback/signal 형태의 비동기 결과로 반환한다.
6. 종료 시 producer(Network/Protocol)를 먼저 중단한 뒤 Core queue 를 drain 하고 recorder 를 flush 한다.

10.2 Backpressure

ATTITUDE 처럼 고주파 이벤트가 UI 보다 훨씬 빠르게 들어올 수 있다. 모든 sample 을 UI queue 에 쌓지 않는다. StateStore 는 최신값을 유지하고 UI 는 20~30 Hz 등 제한된 refresh 주기로 snapshot 을 읽는다. Recorder/Analyzer 만 필요에 따라 모든 sample 을 수신한다. Event 종류별로 Latest-Value, Lossless, Bounded-Queue 정책을 구분한다.

이벤트 유형	정책	예
State telemetry	Latest-value/coalescing	Attitude, position, battery
Command/ACK	Lossless, ordered	Arm ACK, mission ACK
Fault/Health	Lossless + persistence	Failsafe, component fault
Raw log	Bounded high-throughput queue	MAVLink frames
UI notification	Rate-limited	link quality, HUD refresh

11. MAVLink / PX4 / ArduPilot 분리

11.1 왜 두 Adapter 가 필요한가

그림 11-1. Protocol semantics 와 product semantics 의 분리

```
MavlinkProtocolAdapter
  - frame parse/encode
  - CRC/version/signing metadata
  - sysid/compid/message id
  - generic MAVLink microservice mechanics

Px4AutopilotAdapter / ArduPilotAdapter
  - flight-mode semantics
  - vendor-specific parameter meaning
  - capability interpretation
  - command/mode edge cases
  - vendor-specific dialect/extensions
```

이 분리는 "MAVLink 를 지원하는 두 autopilot 은 동일하다"는 잘못된 가정을 방지한다. 같은 MAVLink command 를 사용하더라도 모드, parameter, mission behavior 와 capability 가 달라질 수 있으므로 Product Adapter 가 필요하다.

11.2 초기 MAVLink 지원 범위

도메인	MAVLink 메시지/서비스	KM-GCS canonical target
Discovery	HEARTBEAT	VehicleIdentity/Autopilot type/capability seed
Attitude	ATTITUDE 또는 quaternion 계열	AttitudeState
Position	GLOBAL_POSITION_INT	PositionState + VelocityState
GNSS	GPS_RAW_INT	GnssState
Battery	BATTERY_STATUS / SYS_STATUS 일부	BatteryState/Health
Command	COMMAND_LONG/INT + COMMAND_ACK	VehicleCommandService/CommandResult
Mode	HEARTBEAT base/custom mode + vendor mapping	FlightMode
Mission	MISSION_* microservice	MissionService
Parameter	PARAM_* (후속 EXT/metadata)	ParameterService
Camera/Gimbal	Camera/Gimbal microservices	PayloadService

12. Mission · Parameter · Payload 구조

12.1 Mission semantic layer

그림 12-1. MAVLink waypoint 보다 상위의 Mission Model

```
MissionModel
  MissionTask[]
```

```
    TakeoffTask
    TransitTask
    WaypointTask
    SearchAreaTask
    ObserveTask
    ReturnTask

MissionValidator -> MissionCompiler -> Protocol Mission Adapter

SearchAreaTask
   -> lawnmower/geometric planning
   -> low-level waypoint sequence
   -> MAVLink MISSION_ITEM_INT
```

상위 MissionModel 을 두면 같은 "Search Area"를 PX4/ArduPilot MAVLink mission 으로 compile 하거나, 향후 UCI/DDS 기반 mission command 로 표현할 수 있다. 이것은 protocol-independent mission architecture 의 핵심이다.

12.2 Parameter

ParameterService 는 이름/값만 저장하는 dictionary 가 아니라 type, unit, min/max, default, enum, description, source, read-only 여부를 포함하는 metadata-aware model 을 지향한다. UI 는 parameter protocol 을 직접 알지 않는다.

12.3 Payload

그림 12-2. Payload service abstraction

```
IPayloadService
  id(), type(), capabilities(), status()

ICameraService : IPayloadService
  capture(), startRecording(), stopRecording(), zoom(), focus()

IGimbalService : IPayloadService
  pointAngle(), pointLocation(), setMode()

Adapters
  MAVLinkCameraAdapter / ONVIFAdapter / CustomPayloadAdapter
```

13. Plugin · Capability · Architecture View

13.1 Plugin contract

Plugin 은 단순 shared library 가 아니라 manifest 를 가진 module contract 로 취급한다. manifest 에는 ID, semantic version, 제공 capability, 요구 core/API version, dependency, interface profile 을 기록한다.

예시 13-1. Plugin manifest 개념

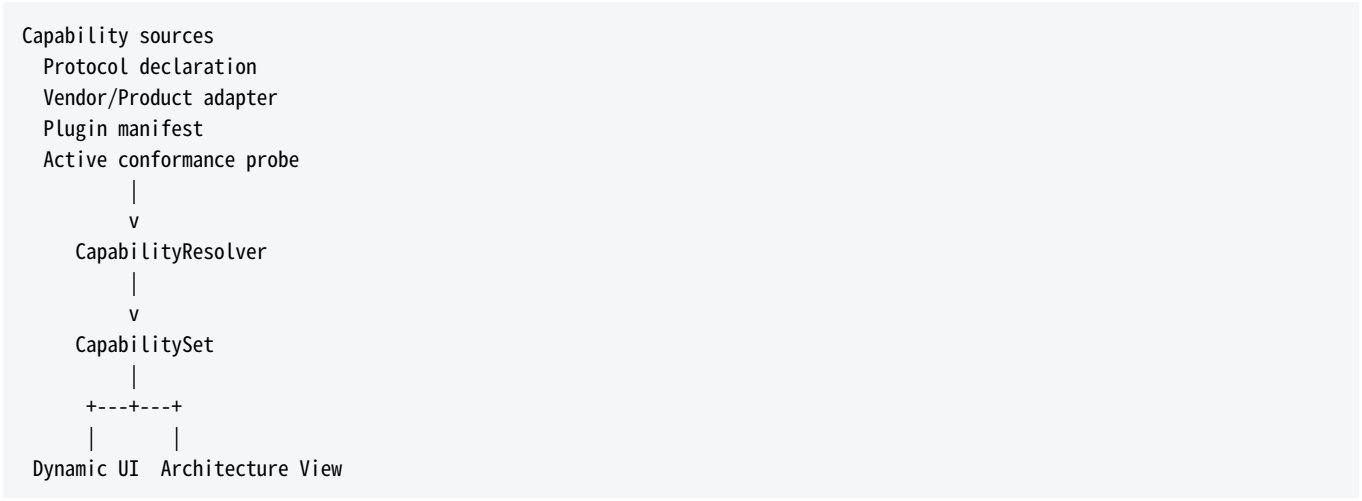
```
{
  "id": "org.kmosa.autopilot.px4",
  "version": "0.1.0",
  "provides": ["autopilot.px4", "mission.mavlink"],
```

```
  "requires": ["kmgcs.core >= 0.1", "protocol.mavlink >= 0.1"],
  "interfaceProfile": "KMOSA-UAV-CORE-0.1"
}
```

13.2 Capability discovery

CapabilityService 는 하나의 bitmap 만 믿지 않는다. MAVLink protocol capability, vendor adapter 판단, plugin manifest, active probe 결과를 합쳐 resolved CapabilitySet 을 만든다. capability 에는 source 와 confidence/verification 상태를 둘 수 있다.

그림 13-1. Capability resolution



13.3 Architecture View

Architecture View 는 연결된 vehicle, FC, mission computer, sensor, payload, link 및 protocol 을 graph 로 표현한다. 각 node/edge 에는 component identity, interface type, protocol version, profile, status, conformance 결과를 표시한다. 단순 예쁜 시각화가 아니라 Interface Manager 의 실시간 표현이다.

14. ROS 2 와 DDS 의 위치

ROS 2 는 MOSA/UCS 자체가 아니며, 본 설계에서는 주로 Mission Computer 의 autonomy/robotics framework 로 위치시킨다. DDS 는 ROS 2 가 사용할 수 있는 middleware 중 하나이자, KM-GCS 가 직접 연결할 수 있는 data-centric middleware 로 본다. 최신 ROS 2 문서에서는 DDS 계열뿐 아니라 Zenoh 같은 middleware 선택도 지원하므로 “ROS 2 = DDS”로 고정하지 않는다.

그림 14-1. ROS 2/DDS 의 권장 위치



```
Video path
  Camera/Encoder ---------------- RTP/RTSP/WebRTC --> GCS Video
  (avoid forcing bulk video through ROS 2 control channel)
```

PX4 는 현재 ROS 2 연계를 위해 uXRCE-DDS 및 Zenoh 선택지를 제공하고, DDS 의 경우 PX4 의 uORB topic 을 companion computer 의 ROS 2/DDS 공간으로 연결할 수 있다. 본 프로젝트는 이 경로를 Mission Computer integration 옵션으로 활용할 수 있지만, KM-GCS Core 자체는 ROS 2 에 종속시키지 않는다.

15. Interface Profile 과 Conformance

15.1 Interface Profile

그림 15-1. Interface Profile 개념

```
KMOSA-UAV-CORE-0.1
  Required
    Vehicle identity / heartbeat
    Position / attitude
    Basic command + ACK
    Time semantics
  Optional
    Mission
    Parameter
    Camera/Gimbal
    FTP / Logging

Profile specifies
  Semantics + required/optional capability
  Protocol binding(s)
  Rate/timing constraints
  Coordinate frame / units
  Version rules
  Conformance test cases
```

15.2 Conformance 의 8 개 축

축	검증 질문	예
Syntax	메시지/필드가 존재하고 decode 가능한가?	ATTITUDE frame 정상
Version	지원 버전과 호환되는가?	Profile 0.1 / dialect version
Semantic	좌표계·단위·상태 의미가 맞는가?	NED/FRD mapping
Timing	필요 update rate 를 만족하는가?	attitude >= 10 Hz
Behavior	명령/상태 전이가 규칙대로인가?	ARM 후 ACK + armed state
Performance	latency/jitter 가 제한 내인가?	command RTT < threshold
Capability	선언 capability 가 실제 동작하는가?	Mission upload probe
Dependency	필수 서비스/모듈 dependency 가 충족되는가?	camera requires gimbal optional/required

16. 오류 처리 · 관측성 · 보안 경계

16.1 오류 모델

오류 범주	처리
LinkError	reconnect policy, endpoint 상태, 사용자에게 연결 원인 표시
ParseError	drop + metric 증가; 반복 발생 시 link/protocol health degradation
SemanticError	frame/unit/range violation 을 domain validation 에서 거부/표시
CommandTimeout	CommandTracker timeout, retry policy 는 command 별 명시
ProtocolNack	canonical CommandResult 로 변환
PluginError	plugin 격리, load 실패가 전체 GCS 종료로 번지지 않게 함
StateStale	각 state 에 age/staleness 를 계산하여 UI/health 에 반영

16.2 관측성

- Structured log: module, vehicle UUID, link ID, protocol, event ID, severity 포함.
- Metrics: packet rate, parse errors, queue depth, event latency, command RTT, state age, dropped/coalesced count.
- Telemetry Recorder 는 raw frame 과 canonical event 를 모두 선택적으로 저장해 translation 문제를 재현할 수 있게 한다.

16.3 보안 경계

초기 MVP 에서 복잡한 보안 기능을 모두 구현하지 않더라도 architecture boundary 는 미리 둔다. Link authentication/encryption, MAVLink signing, plugin trust, command authorization, remote endpoint trust, audit log 는 서로 다른 concerns 로 취급한다. 특히 UI 버튼의 존재가 곧 command 권한을 의미하지 않도록 CommandPolicy/Authorization hook 을 향후 삽입 가능하게 한다.

17. 저장소와 빌드 구조

그림 17-1. 권장 repository layout

```
km-gcs/
  CMakeLists.txt
  cmake/
  apps/
    desktop/
  core/
    application/
    domain/
    event/
    vehicle/
    mission/
    parameter/
    payload/
    capability/
    health/
    logging/
  links/
```

```
    common/
    udp/
    serial/
    tcp/
  protocols/
    common/
    mavlink/
    dds/           # later
    ros2/          # later
    uci/           # later
  adapters/
    autopilot/px4/
    autopilot/ardupilot/
    payload/
  plugins/
  ui/
    shell/ fly/ plan/ setup/ payload/ analyze/ architecture/
  tools/
    interface_manager/
    profile_manager/
    conformance/
  tests/
    unit/ integration/ sitl/
  third_party/
    mavlink/
```

17.1 CMake target 분리

예시 17-1. CMake target 경계

```
kmgcs_domain          # pure C++ domain types, Qt 의존 최소화
kmgcs_event           # event definitions/bus abstractions
kmgcs_core            # registries/services/orchestration
kmgcs_link            # ILink + common
kmgcs_link_udp        # QtNetwork implementation
kmgcs_protocol        # protocol abstractions
kmgcs_mavlink         # MAVLink parser/translator
kmgcs_px4             # PX4 product adapter
kmgcs_ardupilot       # ArduPilot product adapter
kmgcs_ui              # QML module / viewmodels
km-gcs-desktop        # executable
```

가능하면 domain layer 는 Qt QObject 에 과도하게 종속시키지 않는다. Qt signal/slot 이 필요한 경계는 application/viewmodel 또는 EventBus implementation 에서 사용하고, domain value object 는 표준 C++ 타입을 우선한다.

18. MVP 구현 순서

첫 코드는 architecture 를 모두 구현한 뒤 시작하는 것이 아니라, 최소 수직 절단(vertical slice)을 통해 핵심 경계가 실제로 동작하는지를 검증한다. 아래 순서에서는 각 단계가 실행 가능한 상태를 유지한다.

단계	구현	완료 조건
0	Qt6/CMake project, empty shell	Desktop app 가 실행되고 CI/build 가 통과

단계	구현	완료 조건
1	ILink + UdpLink	UDP 14550 datagram 수신/송신 테스트
2	MavlinkProtocolAdapter	HEARTBEAT parse, parse error metric
3	VehicleRegistry	sysid 기반 vehicle 자동 생성/timeout
4	Translator + DomainEvent	HEARTBEAT/ATTITUDE 를 canonical event 로 변환
5	VehicleStateStore	latest state snapshot 유지
6	QML ViewModel	Connected, roll/pitch/yaw 표시
7	Position/GPS/Battery	기본 flight telemetry UI
8	CommandService + PX4/AP adapter	ARM/DISARM + ACK
9	FlightMode mapping	PX4/AP 에서 canonical mode 표시/설정
10	Map View	위치/heading 표시
11	Recorder/metrics	raw/domain logging 및 packet rate 확인
12	SITL dual test	PX4 와 ArduPilot 을 Core 수정 없이 연결

18.1 v0.1 에서 의도적으로 하지 않는 것

- ROS 2/DDS/UCI 구현
- 대규모 Plugin hot-reload
- 전체 QGC 기능 복제
- 복잡한 Mission planner
- Camera/video pipeline
- 완전한 conformance suite
- 다중 vehicle 동시 mission control

이 기능들은 나중에 추가한다. v0.1 의 성공 기준은 "MAVLink 가 Core 로 침투하지 않는 구조를 유지하면서 실제 UAV/SITL 을 연결하고 기본 운용을 수행할 수 있는가"다.

19. 시험 전략

19.1 Unit Test

- MAVLink message → canonical state translation golden test.
- PX4/ArduPilot flight-mode mapping test.
- Reference frame conversion test (NED/ENU, FRD/FLU).
- VehicleRegistry discovery/timeout/duplicate sysid handling.
- State staleness, command timeout, capability resolution.

19.2 Integration Test

- Recorded MAVLink stream replay → StateStore 결과 검증.
- UdpLink ↔ parser ↔ VehicleRegistry 전체 pipeline.
- ARM command encode → synthetic ACK → CommandResult.
- Queue backpressure 에서 UI thread 가 block 되지 않는지 검증.

19.3 SITL/Hardware-in-the-loop

시험	환경	핵심 관찰
PX4 SITL	PX4 + UDP	기본 telemetry/command/mode
ArduPilot SITL	ArduPilot + UDP	동일 Core 와 UI, adapter 만 변경
Dual vehicle	PX4 + AP 동시	VehicleRegistry 와 routing
Physical FC	Kakute/Pixhawk 계열 + UART/UDP bridge	실제 packet loss/timing
HaLow bridge	ESP32-S3/HaLow + MAVLink	link metrics/bandwidth/staleness
Mission computer later	PX4 + ROS 2/DDS	Core 변경 없이 새로운 adapter 삽입

20. 첫 코드 진입용 인터페이스 골격

아래는 상세 설계를 실제 코드로 옮길 때 가장 먼저 만들 파일의 골격이다. API 는 v0.1 구현 중 조정할 수 있지만, protocol type 이 domain/service 경계를 넘어오지 않는다는 원칙은 유지한다.

코드 골격 20-1. Domain value type

```
// core/domain/VehicleId.hpp
struct VehicleId {
    std::array<std::byte, 16> uuid;
};

enum class ReferenceFrame { WGS84, ECEF, NED, ENU, BodyFRD, BodyFLU };

struct AttitudeState {
    Quaternion orientation;
    Vec3 angularVelocity;
    ReferenceFrame bodyFrame{ReferenceFrame::BodyFRD};
    TimePoint sourceTime;
    TimePoint receiveTime;
};
```

코드 골격 20-2. Link abstraction

```
// links/common/ILink.hpp
class ILink {
public:
    virtual ~ILink() = default;
    virtual LinkId id() const = 0;
    virtual void start() = 0;
    virtual void stop() = 0;
    virtual void send(ByteBuffer data) = 0;
    // bytesReceived는 callback/event sink 또는 Qt queued signal로 구현
};
```

코드 골격 20-3. Protocol abstraction

```
// protocols/common/IProtocolAdapter.hpp
class IProtocolAdapter {
public:
```

```
    virtual ~IProtocolAdapter() = default;
    virtual void consume(LinkId link, ByteView bytes, EndpointMeta meta) = 0;
    virtual ByteBuffer encode(const ProtocolCommand& command) = 0;
};
```

코드 골격 20-4. Vehicle use-case API

```
// core/vehicle/IVehicleCommandService.hpp
class IVehicleCommandService {
public:
    virtual ~IVehicleCommandService() = default;
    virtual CommandHandle arm() = 0;
    virtual CommandHandle disarm() = 0;
    virtual CommandHandle takeoff(Meters altitude) = 0;
    virtual CommandHandle land() = 0;
    virtual CommandHandle setFlightMode(FlightMode mode) = 0;
};
```

코드 골격 20-5. Product semantics abstraction

```
// adapters/autopilot/IAutopilotAdapter.hpp
class IAutopilotAdapter {
public:
    virtual ~IAutopilotAdapter() = default;
    virtual ProtocolCommand makeArmCommand(bool arm) const = 0;
    virtual ProtocolCommand makeModeCommand(FlightMode mode) const = 0;
    virtual FlightMode mapFlightMode(const ProductTelemetryView&) const = 0;
    virtual CapabilitySet inferCapabilities(const ProductTelemetryView&) const = 0;
};
```

코드 골격 20-6. Typed Domain Event

```
// core/event/DomainEvent.hpp
using DomainEvent = std::variant<
    VehicleDiscovered,
    VehicleAttitudeUpdated,
    VehiclePositionUpdated,
    GnssUpdated,
    BatteryUpdated,
    CommandAcknowledged,
    LinkQualityUpdated,
    FaultRaised>;
```

20.1 첫 수직 절단의 실제 객체 연결

그림 20-1. 첫 실행 가능한 slice

```
main.cpp
  -> GcsApplication
     -> GcsCore
        -> EventBus
        -> LinkManager
           -> UdpLink(14550)
```

```
        -> ProtocolManager
           -> MavlinkProtocolAdapter
           -> MavlinkTranslator
        -> VehicleRegistry
           -> VehicleContext
              -> VehicleStateStore

QML
  -> GcsViewModel
     -> VehicleRegistry read API
     -> current VehicleState snapshot
```

21. 향후 로드맵

버전/단계	핵심 기능	MOSA 관점 검증
v0.1	기본 MAVLink telemetry/command/map	Protocol leakage 방지
v0.2	Mission + Parameter	Service abstraction
v0.3	Video/Camera/Gimbal	Payload replaceability
v0.4	Plugin + Capability	Dynamic composition
v0.5	Architecture View + Interface Matrix	Architecture-aware control segment
v0.6	Profile + Conformance	검증 가능한 interface contract
v0.7	DDS/ROS2 adapter	Protocol substitution without Core change
v0.8	Multi-vehicle/MUM-T	Heterogeneous system orchestration
v1.0	통합 KM-GCS	K-MOSA 지향 Control Segment baseline

부록 A. 미국 표준/개념과 KM-GCS 대응표

미국 개념	공식 역할 요약	KM-GCS 에서의 해석/대응
DoD MOSA	통합 business/technical strategy, modular/open, verifiable interfaces	전체 설계 원칙 및 replaceability 목표
SAE AS6512B UCS AD	UCS Architecture Library 의 Architecture Description	Control Segment architecture baseline 문서화 방식
SAE AS6518B UCS Model	UCS 모델의 authoritative source	Canonical model + service/interface architecture model
SAE AS6513B Conformance	UCS product conformance requirements	Interface/Profile conformance engine
SAE AS6522B Governance	architecture model/user extensions governance	Plugin/profile/version/extension governance
FACE	avionics SW portability/modularity	API/Plugin 경계와 구현 종속 완화
SOSA	C4ISR sensor architecture	센서/임무처리 subsystem 확장 참고
OMS	services↔subsystems interface/data exchange	Mission/Payload service abstraction
UCI	mission-level machine-to-machine C2	향후 mission/C2 adapter
ROS 2	미국 MOSA 표준은 아님; robotics framework	Mission Computer autonomy framework

미국 개념	공식 역할 요약	KM-GCS 에서의 해석/대응
DDS	data-centric middleware	직접 DDS adapter 및 ROS2 RMW transport option
MAVLink	오픈 UAS messaging/microservices	초기 vehicle-level protocol binding

부록 B. 공개 참고 자료

[R1] U.S. Department of Defense, Systems Engineering & Architecture, “Modular Open Systems Approach (MOSA),” cto.mil/sea/mosa/.

[R2] U.S. DoD, “Implementing a Modular Open Systems Approach in Department of Defense Programs,” MOSA Implementation Guidebook, Feb. 2025.

[R3] NAVAIR, “MOSA,” overview of HOST, FACE, SOSA and OMS.

[R4] SAE International, AS6512B, “Unmanned Systems (UxS) Control Segment (UCS) Architecture: Architecture Description,” revised 2022-03-16.

[R5] SAE International, AS6518B, “UCS Architecture Model,” revised 2022-03-16.

[R6] SAE International, AS6513B, “Conformance Specification,” revised 2022-03-16.

[R7] SAE International, AS6522B, “Architecture Technical Governance,” stabilized 2024-02-27.

[R8] SAE International, AIR6520A, UCS Architecture Version Description Document, stabilized 2024-02-23; governance transfer from OSD to SAE noted.

[R9] K-MOSA Research Center, “About” and “Research,” 2026.

[R10] ROS 2 Documentation, “Different ROS 2 middleware vendors,” current documentation; DDS/RTPS and Zenoh middleware options.

[R11] PX4 User Guide, ROS 2 integration / uXRCE-DDS documentation; PX4 uORB–ROS 2 integration and middleware options.

결론: 구현 직전의 Architecture Baseline

KM-GCS 의 첫 구현은 “QGC 같은 기능을 빠르게 복제”하는 것이 목적이 아니다. 첫 목표는 Link→Protocol→Translation→Domain Event→Vehicle State→Service→UI 라는 경계를 실제 코드에서 유지하면서 PX4 와 ArduPilot 을 연결하는 것이다. 이 경계가 지켜져야 이후 ROS 2/DDS, Payload, Interface Profile, Conformance 기능을 Core 재설계 없이 확장할 수 있다.

따라서 다음 작업은 이 문서를 기준으로 repository 를 생성하고, kmgcs_domain → kmgcs_link → kmgcs_mavlink → kmgcs_core → kmgcs_ui 순으로 최소 target 을 만들며, PX4 SITL HEARTBEAT 와 ATTITUDE 를 수신해 QML 에 표시하는 첫 수직 절단을 구현하는 것이다. 그 다음 동일 Core 에 ArduPilot SITL 을 붙여 “Autopilot replacement without GCS core modification”의 최초 검증을 수행한다.